

UD01.05: Tipos de datos

Contenido

1. Introducción a los tipos de datos en Python.....	3
1.1 Tipado dinámico y fuerte en Python	3
1.2 Identificando Tipos de Datos con type()	3
1.3 Conversión de tipos (castings).....	4
2. Tipos numéricos	5
2.1 Enteros (int)	5
2.2 Números de punto flotante (float)	5
2.3 Números complejos (complex).....	5
2.4 Operaciones básicas con tipos numéricos	5
3. Booleanos (bool)	6
3.1 Tipo bool.....	6
3.2 Valores True y False	6
3.3 Operadores lógicos y su relación con booleanos	6
4. Secuencias	7
4.1 Cadenas de caracteres (str)	8
4.2 Listas (list)	8
4.3 Tuplas (tuple)	8
4.4 Indexación, <i>slicing</i> y operaciones comunes en secuencias	8
5. Conjuntos	10
5.1 Conjuntos mutables (set).....	10
5.2 Conjuntos inmutables (frozenset).....	10
5.3 Operaciones con conjuntos: unión, intersección, diferencia	10
6. Diccionarios (dict)	12
6.1 Concepto de clave-valor (dict).....	12
6.2 Métodos básicos para manipular diccionarios	12
6.3 Uso común y ejemplos prácticos.....	12
7. Tipo None	13
7.1 NoneType y su significado	13
7.2 Uso habitual en programas	13

8. Métodos para Manipular Tipos de Datos.....	14
8.1 Métodos de Cadenas (str)	14
8.2 Métodos de Listas (list)	15
8.3 Métodos de Diccionarios (dict)	16

1. Introducción a los tipos de datos en Python

En Python, cada valor tiene un tipo de dato asociado que define las operaciones que se pueden realizar con él. Comprender los tipos de datos es fundamental para escribir programas correctos y eficientes.

1.1 Tipado dinámico y fuerte en Python

- **Tipado dinámico:** Python es un lenguaje de **tipado dinámico**. Esto significa que no necesitas declarar el tipo de una variable antes de asignarle un valor. El intérprete determina el tipo de la variable en tiempo de ejecución.
- **Tipado fuerte:** Python es un lenguaje de **tipado fuerte**. Esto implica que no se realizan conversiones de tipo implícitas entre tipos no relacionados. Por ejemplo, no puedes sumar un int y un str directamente; el programa te obligará a realizar una conversión explícita.

```
x = 10    # x es de tipo int
```

```
x = "Hola"  # x ahora es de tipo str
```

```
# Esto provocaría un error (TypeError)
```

```
# print("Mi edad es: " + 25)
```

```
# El tipo 'str' y 'int' no son compatibles para la concatenación
```

1.2 Identificando Tipos de Datos con type()

Python proporciona una función incorporada llamada `type()` que te permite conocer el tipo de dato de cualquier valor o variable. Es una herramienta esencial para la depuración y para entender cómo se comporta el intérprete con los diferentes tipos de datos.

Sintaxis:

```
type(variable_o_valor)
```

Ejemplos:

```
mi_numero = 10
```

```
print(type(mi_numero))    # Salida: <class 'int'>
```

```
mi_cadena = "hola"
```

```
print(type(mi_cadena))    # Salida: <class 'str'>
```

```
mi_lista = [1, 2, 3]
```

```
print(type(mi_lista))     # Salida: <class 'list'>
```

1.3 Conversión de tipos (castings)

Para convertir un tipo de dato a otro, se utilizan funciones de conversión explícita (también conocidas como *casting*).

- `int(valor)`: Convierte el valor a un número entero.
- `float(valor)`: Convierte el valor a un número de punto flotante.
- `str(valor)`: Convierte el valor a una cadena de texto.
- `bool(valor)`: Convierte el valor a un booleano.

```
edad_str = "25"
```

```
edad_int = int(edad_str)    # Convierte "25" a 25
```

```
print(f"Tipo de edad_int: {type(edad_int)}")
```

```
precio = 19.99
```

```
precio_str = str(precio)    # Convierte 19.99 a "19.99"
```

```
print(f"Tipo de precio_str: {type(precio_str)}")
```

2. Tipos numéricos

Python maneja tres tipos principales de números.

2.1 Enteros (int)

Representan números enteros, positivos o negativos, sin decimales. Los enteros en Python no tienen un límite de tamaño, pueden ser tan grandes como lo permita la memoria del sistema.

```
a = 100
```

```
b = -50
```

```
numero_grande = 12345678901234567890
```

2.2 Números de punto flotante (float)

Representan números reales con una parte decimal.

```
pi = 3.14159
```

```
temperatura = 25.5
```

```
notacion_cientifica = 1.2e-5 # Representa 0.000012
```

2.3 Números complejos (complex)

Representan números complejos en el formato $a + bj$, donde a es la parte real y b es la parte imaginaria.

```
z = 3 + 4j
```

```
print(f"Parte real: {z.real}")
```

```
print(f"Parte imaginaria: {z.imag}")
```

2.4 Operaciones básicas con tipos numéricos

Los tipos numéricos admiten operadores aritméticos como $+$, $-$, $*$, $/$, $//$, $%$ y $**$.

3. Booleanos (bool)

3.1 Tipo bool

El tipo bool representa valores de verdad. Solo tiene dos posibles valores.

3.2 Valores True y False

- True: Representa una condición verdadera.
- False: Representa una condición falsa.

```
mayor_de_edad = (edad >= 18) # Esto devuelve True o False
```

```
es_activo = True
```

3.3 Operadores lógicos y su relación con booleanos

Los operadores lógicos (and, or, not) trabajan con valores booleanos.

- and: Devuelve True si ambos operandos son True.
- or: Devuelve True si al menos uno de los operandos es True.
- not: Invierte el valor del operando.

```
condicion1 = (5 > 3) # True
```

```
condicion2 = (10 == 9) # False
```

```
print(condicion1 and condicion2) # False
```

```
print(condicion1 or condicion2) # True
```

```
print(not condicion1) # False
```

4. Secuencias

Las secuencias son colecciones de elementos ordenados. Python tiene varios tipos de secuencias, siendo las más comunes las cadenas, las listas y las tuplas.

Una de las distinciones más importantes en Python es si un tipo de dato es **mutable** o **inmutable**.

- **Inmutable:** El valor del objeto no puede ser modificado después de su creación. Si intentas "cambiarlo", en realidad se crea un nuevo objeto en la memoria.
 - **Tipos inmutables:** Enteros (int), flotantes (float), booleanos (bool), cadenas (str), tuplas (tuple), números complejos (complex), y conjuntos inmutables (frozenset).
- **Mutable:** El valor del objeto puede ser modificado en su lugar, sin necesidad de crear un nuevo objeto en la memoria.
 - **Tipos mutables:** Listas (list), conjuntos (set), y diccionarios (dict).

Ejemplo de Mutabilidad:

```
# Lista (mutable)
```

```
mi_lista = [1, 2, 3]
```

```
print(f"ID antes: {id(mi_lista)}")
```

```
mi_lista.append(4)
```

```
print(f"ID después: {id(mi_lista)}")
```

```
# Los IDs son iguales, el objeto no se ha movido en la memoria.
```

```
# Tupla (inmutable)
```

```
mi_tupla = (1, 2, 3)
```

```
print(f"ID antes: {id(mi_tupla)}")
```

```
# Esto genera un error: mi_tupla.append(4)
```

4.1 Cadenas de caracteres (str)

Son secuencias de caracteres **inmutables**.

```
saludo = "Hola, mundo"

print(saludo[0])    # Accede al primer carácter: 'H'

print(len(saludo))  # Longitud de la cadena: 11
```

4.2 Listas (list)

Son colecciones ordenadas y **mutables** (se pueden modificar) de elementos. Se definen con corchetes [].

```
numeros = [10, 20, 30, 40]

numeros.append(50) # Añade un elemento

print(numeros)    # [10, 20, 30, 40, 50]
```

4.3 Tuplas (tuple)

Son colecciones ordenadas e **inmutables** (no se pueden modificar) de elementos. Se definen con paréntesis ().

```
coordenadas = (10, 20)

print(coordenadas[0]) # Accede al primer elemento: 10

# Esto provocaría un error (TypeError)

# coordenadas.append(30)
```

4.4 Indexación, *slicing* y operaciones comunes en secuencias

- **Indexación:** Acceder a un elemento de la secuencia mediante su posición (índice), que comienza en 0. Se puede usar indexación negativa (-1 para el último elemento).
- **Slicing:** Extraer una sub-secuencia. Se usa el formato [inicio:fin:paso].
- **Operaciones:** Concatenación (+), repetición (*), y operadores de pertenencia (in).

```
mi_lista = ['a', 'b', 'c', 'd', 'e']  
print(mi_lista[1])    # 'b'  
print(mi_lista[-1])   # 'e' (el último)  
print(mi_lista[1:4])  # ['b', 'c', 'd'] (desde el índice 1 hasta el 3)
```

5. Conjuntos

Los conjuntos son colecciones de elementos **no ordenados** y **sin duplicados**. Son útiles para operaciones de pertenencia y para eliminar duplicados.

5.1 Conjuntos mutables (set)

Se definen con llaves {}. Se pueden añadir y eliminar elementos.

```
numeros = {1, 2, 3, 4, 4}
print(numeros) # Salida: {1, 2, 3, 4} (se eliminó el duplicado)
numeros.add(5)
numeros.remove(1)
```

5.2 Conjuntos inmutables (frozenset)

Son como los conjuntos, pero no se pueden modificar una vez creados. Son útiles cuando necesitas que un conjunto sea la clave de un diccionario, ya que las claves deben ser inmutables.

```
fs = frozenset([1, 2, 3])
# fs.add(4) # Error: frozenset no tiene el método add
```

5.3 Operaciones con conjuntos: unión, intersección, diferencia

Se pueden realizar operaciones matemáticas de conjuntos.

- **Unión:** |
- **Intersección:** &
- **Diferencia:** -
- **Diferencia simétrica:** ^

```
a = {1, 2, 3}
```

```
b = {3, 4, 5}
```

```
print(f"Unión: {a | b}")    # {1, 2, 3, 4, 5}
```

```
print(f"Intersección: {a & b}") # {3}
```

6. Diccionarios (dict)

6.1 Concepto de clave-valor (dict)

Un diccionario es una colección de pares **clave-valor**. Es una colección no ordenada (desde Python 3.7 son ordenados) y mutable. Las claves deben ser únicas e inmutables.

```
persona = {  
    "nombre": "Ana",  
    "edad": 25,  
    "ciudad": "Madrid"  
}
```

```
print(persona["nombre"]) # Acceder a un valor usando su clave: 'Ana'
```

6.2 Métodos básicos para manipular diccionarios

- `diccionario.get(clave)`: Obtiene un valor de forma segura, devolviendo `None` si la clave no existe.
- `diccionario[clave] = valor`: Asigna o modifica un valor.
- `diccionario.keys()`: Devuelve las claves del diccionario.
- `diccionario.values()`: Devuelve los valores del diccionario.
- `diccionario.items()`: Devuelve pares (clave, valor).

```
persona["edad"] = 26
```

```
print(persona) # {'nombre': 'Ana', 'edad': 26, 'ciudad': 'Madrid'}
```

6.3 Uso común y ejemplos prácticos

Los diccionarios son ideales para representar datos estructurados como un registro de una base de datos o los datos de un usuario.

7. Tipo None

7.1 NoneType y su significado

El tipo NoneType solo tiene un valor posible: None. Este valor se utiliza para representar la ausencia de valor.

7.2 Uso habitual en programas

- **Inicializar una variable:** Cuando se necesita una variable pero aún no se le ha asignado un valor real.
- **Valores de retorno de funciones:** Una función que no devuelve nada explícitamente, en realidad devuelve None.
- **Como valor por defecto:** En parámetros de funciones para indicar que un argumento es opcional.

Función que puede o no devolver un valor

```
def buscar_usuario(id):
```

```
    if id == 123:
```

```
        return {"id": 123, "nombre": "Juan"}
```

```
    else:
```

```
        return None # Devuelve None si no se encuentra el usuario
```

```
usuario = buscar_usuario(456)
```

```
if usuario is None:
```

```
    print("Usuario no encontrado.")
```

8. Métodos para Manipular Tipos de Datos

Además de las operaciones básicas, muchos tipos de datos en Python tienen **métodos** integrados que permiten realizar acciones específicas. A diferencia de las funciones, los métodos se llaman utilizando la notación de punto (.) en una instancia del objeto.

8.1 Métodos de Cadenas (str)

Las cadenas son **inmutables**, lo que significa que sus métodos no las modifican, sino que **devuelven una nueva cadena** con el resultado de la operación.

- **.upper()** y **.lower()**: Devuelven una copia de la cadena con todos sus caracteres convertidos a mayúsculas o minúsculas.

Ejemplo:

```
saludo = "Hola Mundo"
```

```
minusculas = saludo.lower() # Devuelve "hola mundo"
```

```
mayusculas = saludo.upper() # Devuelve "HOLA MUNDO"
```

- **.strip()**: Devuelve una copia de la cadena sin los espacios en blanco iniciales y finales.

Ejemplo:

```
texto = " Python es genial "
```

```
limpio = texto.strip() # Devuelve "Python es genial"
```

- **.replace(old, new)**: Devuelve una copia de la cadena con todas las ocurrencias de una subcadena reemplazadas por otra.

Ejemplo:

```
frase = "Me gusta el color azul"
```

```
nueva_frase = frase.replace("azul", "rojo") # Devuelve "Me gusta el color rojo"
```

- **.split(separador)**: Devuelve una **lista** de subcadenas, dividiendo la cadena original por el separador especificado.

Ejemplo:

```
colores = "rojo,verde,azul"
```

```
lista_colores = colores.split(",") # Devuelve ['rojo', 'verde', 'azul']
```

8.2 Métodos de Listas (list)

Las listas son **mutables**, por lo que la mayoría de sus métodos **modifican la lista directamente**.

- **.append(elemento)**: Añade un único elemento al final de la lista.

Ejemplo:

```
numeros = [1, 2, 3]
```

```
numeros.append(4) # La lista ahora es [1, 2, 3, 4]
```

- **.insert(indice, elemento)**: Inserta un elemento en la posición del índice especificado.

Ejemplo:

```
letras = ['a', 'c', 'd']
```

```
letras.insert(1, 'b') # La lista ahora es ['a', 'b', 'c', 'd']
```

- **.remove(valor)**: Elimina la **primera aparición** del valor especificado de la lista. Si el valor no existe, produce un error.

Ejemplo:

```
numeros = [10, 20, 30, 20]
```

```
numeros.remove(20) # La lista ahora es [10, 30, 20]
```

- **.pop(indice)**: Elimina y **devuelve** el elemento en el índice especificado. Si no se especifica un índice, elimina y devuelve el último elemento.

Ejemplo:

```
pila = [1, 2, 3]
```

```
ultimo = pila.pop() # ultimo es 3, y pila es [1, 2]
```

- **.sort():** Ordena los elementos de la lista en su lugar (modifica la lista directamente).

Ejemplo:

```
numeros = [3, 1, 2]
```

```
numeros.sort() # La lista ahora es [1, 2, 3]
```

8.3 Métodos de Diccionarios (dict)

Los diccionarios son **mutables** y sus métodos se utilizan para acceder, manipular y recorrer los pares clave-valor.

- **.keys():** Devuelve un "objeto vista" que muestra todas las claves del diccionario.
- **.values():** Devuelve un "objeto vista" que muestra todos los valores del diccionario.
- **.items():** Devuelve un "objeto vista" con los pares (clave, valor) del diccionario.

Ejemplo:

```
persona = {"nombre": "Ana", "edad": 25}
```

```
print(persona.keys()) # Salida: dict_keys(['nombre', 'edad'])
```

```
print(persona.values()) # Salida: dict_values(['Ana', 25])
```

```
print(persona.items()) # Salida: dict_items([('nombre', 'Ana'), ('edad', 25)])
```

- **.get(clave, valor_por_defecto):** Devuelve el valor asociado a la clave. Si la clave no existe, no produce un error, sino que devuelve None o el valor_por_defecto que le pases.

Ejemplo:

```
telefono = persona.get("telefono", "No disponible") # Devuelve "No disponible"
```

```
nombre = persona.get("nombre") # Devuelve "Ana"
```